SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3 – TRANSFORMER MECHANICS WORKSHEET

Course Code:	CSA6506	**Course Title:**	Generative AI and Large Language Models
CO Assessed:	CO1 – Precision (BL1, BL2)	**Assessment:**	Assessment Tool 3 – Transformer Mechanics Worksheet
Weightage:	40% of CIA	**Total Marks:**	100
CO1 Statement:	*Explain the fundamentals of Artificial Intelligence, Generative AI, Foundation Models, Transformer architecture, tokenization, embeddings, and Large Language Models. (BL1, BL2)*		
Student Name:	**T Bhanu Teja**	**Reg. No.:**	**192472259**
Section / Batch:	2024(CSE-AI)	**Date:**	14.10.26

Instructions to Students

• Answer all questions. It carries 100 Marks. Total: 100 Marks.

· Tokenize a sentence.
· Convert tokens to token IDs.
· Explain embeddings.
· Label a Transformer encoder.
· Identify Query, Key, and Value.
· Explain which words receive higher attention.
· Compare Transformer models.
· Complete a Transformer pipeline.
· Draw a block diagram.

Code of Conduct

I ___T Bhanu Teja/192472259___ *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: _ Bhanu Teja__

Question 1: Sentence Tokenization

Consider the sentence: “Transformers enable intelligent text generation.”

Split the sentence into tokens using a suitable tokenizer. Explain how the tokenization may differ between a word-level tokenizer and a subword tokenizer.

Topic: Tokenization | Bloom’s level: Understand

Question 2: Token-to-ID Conversion

Given the token sequence `["AI", "models", "learn"]` and the vocabulary mapping `{"AI": 12, "models": 25, "learn": 8}`, convert the tokens into their corresponding token IDs. What happens if a token is not present in the vocabulary?

Topic: Token IDs | Bloom’s level: Apply

Question 3: Understanding Embeddings

Explain how token IDs are converted into embedding vectors in a Transformer. Why are embedding vectors more useful for language processing than token IDs alone?

Topic: Token embeddings | Bloom’s level: Understand

Question 4: Transformer Encoder Labelling

Draw a Transformer encoder block and label the following components in the correct order: multi-head self-attention, input embeddings with positional information, feed-forward neural network, residual connections, and layer normalization.

Topic: Transformer encoder architecture | Bloom’s level: Apply

Question 5: Query, Key, and Value Identification

In the sentence “The student submitted the assignment because it was complete,” explain how Query, Key, and Value vectors are generated from token representations. Describe their respective roles in determining relationships between words.

Topic: Query–Key–Value | Bloom’s level: Analyze

Question 6: Attention Weight Analysis

Consider the sentence: “The cat chased the mouse because it moved quickly.” Which words might receive higher attention when processing the word “it”? Justify your answer and explain how attention weights help a Transformer capture contextual relationships.

Topic: Self-attention | Bloom’s level: Analyze

Question 7: Comparing Transformer Models

Compare BERT, GPT, and T5 in terms of their attention approach, input-output behaviour, and typical applications. Identify which model would be most suitable for (a) text classification, (b) open-ended text generation, and (c) text summarization, with justification.

Topic: Transformer model comparison | Bloom's level: Analyze

Question 8: Completing the Transformer Pipeline

Complete the following Transformer processing pipeline by filling in the missing stages:

Text Input → _____ → _____ → Embedding and Positional Information → _____ → Output Probabilities → Predicted Token.

Explain the function of each missing stage.

Topic: Transformer pipeline | Bloom's level: Apply

Question 9: Transformer Block Diagram

Draw a block diagram illustrating how an input sentence is processed by a Transformer-based language model to generate an output. Include tokenization, token ID conversion, embeddings, attention layers, feed-forward layers, output projection, and token prediction.

Topic: Transformer architecture | Bloom's level: Create

Question 10: Integrated Transformer Mechanics

A language model receives the sentence "Generative AI creates new content." Trace the sentence through tokenization, token ID conversion, embedding generation, self-attention, contextual representation, and output prediction. Explain how the attention mechanism helps the model understand relationships among the words.

Topic: Integrated Transformer pipeline | Bloom's level: Analyze

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Mechanics Explanation	30%	All key concepts (AI, ML, DL, GenAI, LLMs) correctly identified with clear hierarchical relationships	Most concepts correct; minor gaps in relationships	Several concepts missing or misclassified	Major conceptual errors; map incomplete
Worked-Example Accuracy	25%	Real-world examples linked accurately to each concept	Examples mostly relevant	Few or generic examples	No real-world linkage shown
Mathematical Reasoning	20%	Logical, well-organized structure with clear connectors	Mostly organized; minor structural issues	Some disorganization affecting clarity	Poorly structured, hard to follow
Diagram & Working Clarity	15%	Visually clear, creative, easy to interpret	Neat and readable	Cluttered or inconsistent formatting	Untidy, difficult to interpret
Step Justification	10%	Concise labels/notes that add clarity	Adequate labeling	Minimal or unclear labeling	No supporting labels or explanation

Marks Evaluation:

Mechanics Explanation (30%)	Worked-Example Accuracy(25%)	Mathematical Reasoning (20%)	Diagram & Working Clarity (15%)	Step Justification (10%)	Total Marks (100)

SOLUTION:

TRANSFORMER MECHANICS WORKSHEET

Question 1: Sentence Tokenization

Topic: Tokenization | **Bloom's Level:** Understand

1. Tokenization

Definition: Tokenization is the process of dividing a sentence into smaller units called tokens. These tokens may be words, subwords, or punctuation marks.

Given sentence:

"Transformers enable intelligent text generation."

Word-level tokenization:

The sentence is divided into individual words, with punctuation separated.

Tokens:

["Transformers", "enable", "intelligent", "text", "generation", "."]

Explanation:

- Each word becomes a separate token.
- The full stop is treated as a separate token.
- The tokens are processed further by the Transformer model.

2. Convert Tokens to Token IDs

Each token is assigned a numerical ID based on the tokenizer's vocabulary.

For illustration, consider the following sample vocabulary mapping:

Token	Sample Token ID
Transformers	101
enable	205
intelligent	312
text	450
generation	521
.	102

Token ID sequence:

[101, 205, 312, 450, 521, 102]

Note: These are illustrative IDs, not the actual IDs from a specific tokenizer. Actual IDs depend on the vocabulary used.

3. Explain Embeddings

Definition: Embeddings are numerical vectors that represent tokens in a form that a Transformer can process.

Working:

1. The tokenizer converts each token into a token ID.
2. The token ID identifies a row in the model's embedding matrix.
3. The corresponding row provides the token's embedding vector.
4. Positional information is added to represent the order of tokens.
5. Transformer layers process these representations to understand the context.

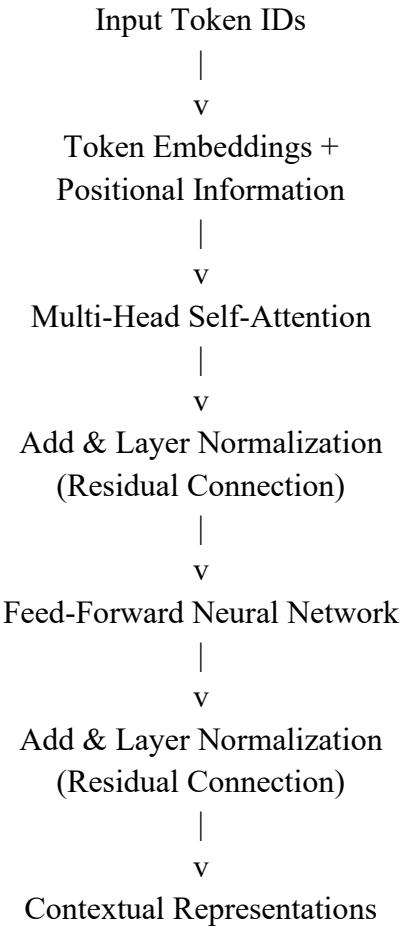
Example: The embedding for *intelligent* represents learned information about that token. After self-

attention, its contextual representation can incorporate information from words such as *Transformers* and *generation*.

4. Label a Transformer Encoder

A Transformer encoder processes token representations using self-attention and feed-forward neural networks.

Labelled block diagram:



Explanation of components:

- **Embeddings:** Convert token IDs into vectors.
- **Positional information:** Represents the order of tokens.
- **Multi-head self-attention:** Helps tokens gather relevant information from other tokens.
- **Residual connections:** Add the input of a sublayer to its output.
- **Layer normalization:** Helps stabilize the representations.
- **Feed-forward network:** Further transforms each token's representation.
- **Contextual representations:** Represent tokens in the context of the complete sentence.

5. Identify Query, Key, and Value (Q, K, V)

In self-attention, each input token representation is transformed into three vectors.

Component	Meaning	Function
Query (Q)	What a token is looking for	Determines which other tokens may be relevant
Key (K)	Information used for matching	Helps determine the relevance of other tokens
Value (V)	Information carried by a token	Supplies information that is combined using attention weights

Example: When processing *generation*, the Query may match Keys associated with *text* because the two words form a meaningful phrase. Their Value vectors contribute to the resulting contextual representation.

6. Explain Which Words Receive Higher Attention

Attention allows the model to determine how much information each token receives from other tokens. Consider:

"Transformers enable intelligent text generation."

Possible relationships include:

- **Text ↔ generation:** These words are related because they form the phrase *text generation*.
- **Transformers ↔ enable:** These words are related because Transformers perform the action described by *enable*.
- **Intelligent ↔ generation:** These words may be related because *intelligent* describes the type of text generation.

For example, when processing *generation*, the model may assign relatively high attention to *text* because the words are closely related.

How attention works:

1. Calculate Query–Key similarity scores.
2. Apply softmax to convert the scores into attention weights.
3. Multiply the attention weights by the Value vectors.
4. Combine the weighted Values to produce a contextual representation.

Important: These are plausible linguistic relationships, not measured attention weights. Actual attention depends on the model, layer, attention head, and input.

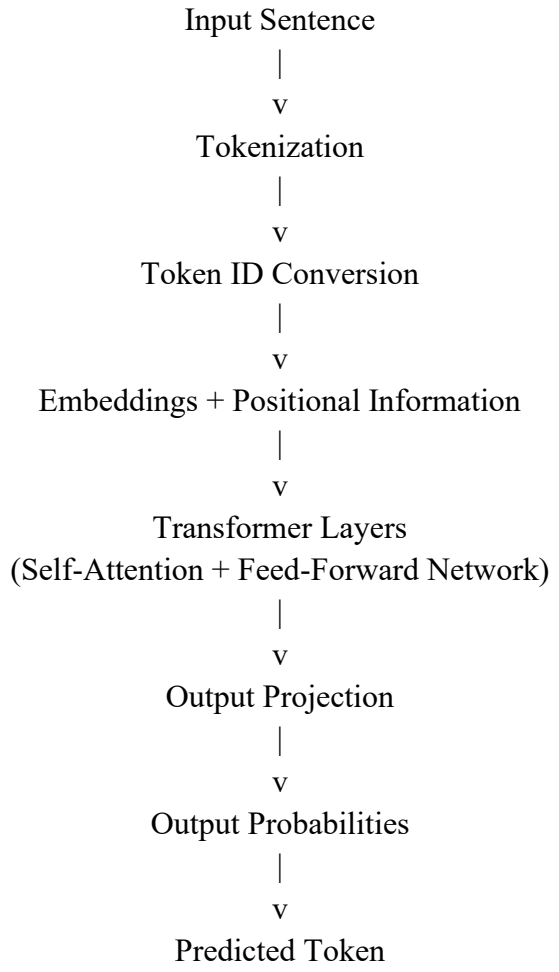
7. Compare Transformer Models

Feature	BERT	GPT	T5
Architecture	Encoder-only	Decoder-only	Encoder–decoder
Attention	Bidirectional self-attention	Causal self-attention	Encoder self-attention, decoder causal attention, and cross-attention
Main purpose	Understand text	Generate text	Transform input text into output text
Applications	Classification and question answering	Text generation and dialogue	Summarization and translation

Relationship to tokenization: All three models tokenize text and convert tokens into numerical representations. However, their architectures and attention mechanisms differ.

8. Complete the Transformer Pipeline

The processing pipeline for a Transformer-based language model can be represented as follows:



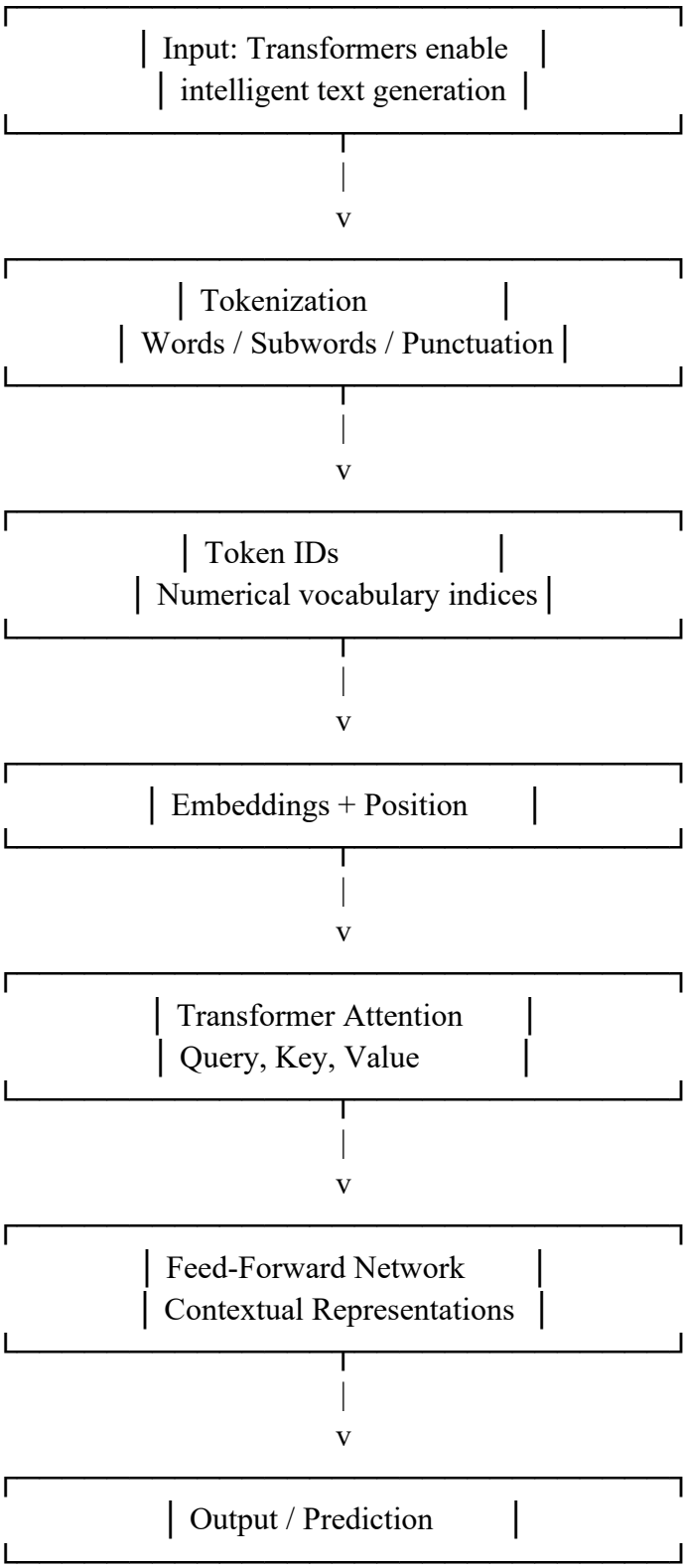
Explanation:

1. **Input sentence:** Receives the original text.
2. **Tokenization:** Divides the text into tokens.
3. **Token ID conversion:** Maps tokens to vocabulary indices.
4. **Embeddings:** Converts token IDs into vectors and incorporates positional information.
5. **Transformer layers:** Learn contextual relationships between tokens.
6. **Output projection:** Converts the final representation into vocabulary scores.
7. **Output probabilities:** Represents the probability assigned to each possible output token.
8. **Predicted token:** Selects a token according to the model's decoding method.

Note: This pipeline describes a generative, decoder-style Transformer. Encoder-only models such as BERT typically use their contextual representations for tasks such as classification.

9. Draw a Block Diagram

The following diagram summarizes the complete process.



Word-Level vs. Subword Tokenization

The main difference is the size of the units produced by the tokenizer.

Word-level tokenization	Subword tokenization
Splits text into complete words and punctuation.	Splits text into words or smaller word parts.
May struggle with rare or unseen words.	Can represent rare words using familiar subword units.
Usually produces fewer tokens for ordinary sentences.	May produce more tokens when words are divided into smaller parts.
Example: ["Transformers", "enable"]	Example: ["transform", "##ers", "enable"]

The subword example is illustrative; the actual division depends on the tokenizer's vocabulary.

Conclusion

Tokenization is an important first step in Transformer processing because it converts text into manageable units. Token IDs provide numerical references to those units, while embeddings represent them as vectors. Self-attention uses Query, Key, and Value vectors to capture relationships between words, allowing Transformer models to build contextual representations for language understanding and generation.

Question 2: Token-to-ID Conversion

Topic: Token IDs | **Bloom's Level:** Apply

1. Convert Tokens to Token IDs

Definition: Token IDs are numerical values assigned to tokens based on a tokenizer's vocabulary. They allow Transformer models to process text numerically.

Given:

- **Tokens:** ["AI", "models", "learn"]
- **Vocabulary:** {"AI": 12, "models": 25, "learn": 8}

Conversion:

Token	Token ID
AI	12
models	25
learn	8

Resulting Token ID Sequence:

[12, 25, 8]

2. What Happens if a Token Is Missing?

If a token is not present in the vocabulary, the tokenizer may:

- **Subword tokenization:** Divide it into smaller, known tokens.
- **Unknown token:** Replace it with a special token such as [UNK], depending on the tokenizer.

3. Explain Embeddings

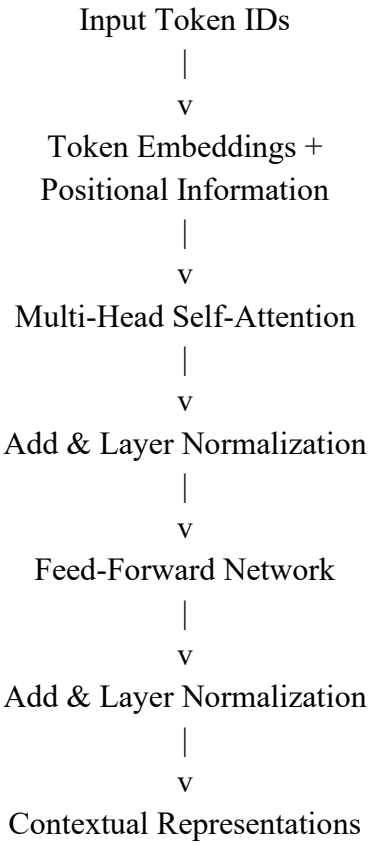
Embeddings convert token IDs into dense numerical vectors using a learned embedding matrix.

Example:

Token ID (12) → Embedding Matrix → Numerical Vector

These vectors represent learned token information. Positional information is also incorporated to represent token order.

4. Label a Transformer Encoder



5. Identify Query, Key, and Value (Q, K, V)

Component	Function
Query (Q)	Searches for relevant information.
Key (K)	Helps determine the relevance of other tokens.
Value (V)	Carries information used to create contextual representations.

The token embeddings are transformed into Q, K, and V vectors through learned projections.

6. Explain Which Words Receive Higher Attention

In the phrase **"AI models learn,"** the word *models* may attend to *AI* because the words are closely related. Attention scores determine how much information one token receives from other tokens. Higher attention weights give the corresponding tokens greater influence on the resulting representation.

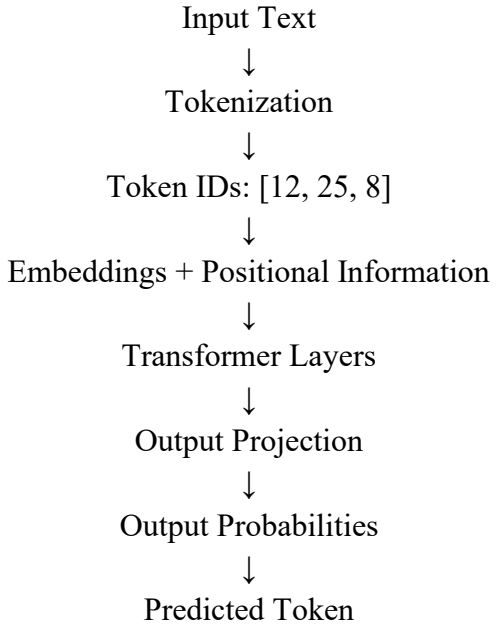
Note: This is an illustrative example, not a measured attention score.

7. Compare Transformer Models

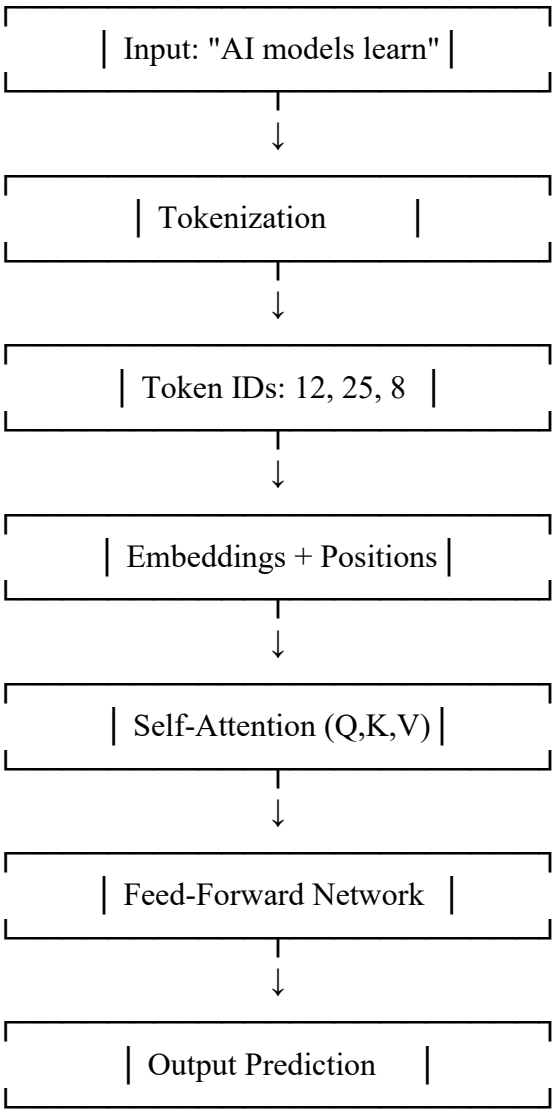
Model	Architecture	Main Application
BERT	Encoder-only	Text classification
GPT	Decoder-only	Text generation
T5	Encoder–decoder	Summarization and translation

All three use token IDs and embeddings, but their architectures and attention mechanisms differ.

8. Complete the Transformer Pipeline



9. Block Diagram



Conclusion

Token-to-ID conversion maps tokens to numerical vocabulary indices. These IDs are converted into embeddings that enable Transformer models to process text. Self-attention then helps the model learn contextual relationships between tokens.

Question 3: Understanding Embeddings

Topic: Token Embeddings | **Bloom's Level:** Understand

1. Explain Embeddings

Definition: Embeddings are dense numerical vectors that represent tokens in a form that a Transformer can understand and process.

How token IDs are converted into embeddings:

1. **Tokenization:** The input sentence is divided into tokens.
2. **Token ID conversion:** Each token is assigned a numerical ID.
3. **Embedding lookup:** The token ID selects a corresponding vector from a learned embedding matrix.
4. **Positional information:** Positional information is added to represent the order of tokens.
5. **Transformer processing:** The vectors pass through Transformer layers to capture contextual relationships.

Example:

Sentence: "AI models learn"

Token	Token ID	Illustrative embedding
AI	12	[0.2, 0.5, 0.1]
models	25	[0.7, 0.3, 0.4]
learn	8	[0.6, 0.8, 0.2]

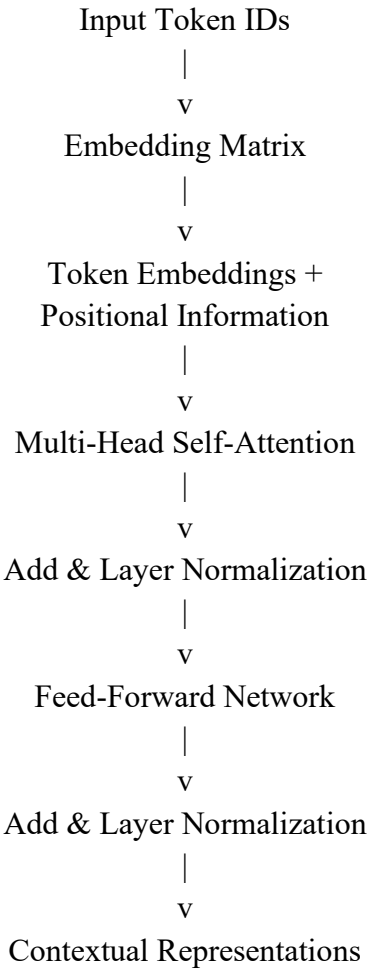
Note: The embedding values are illustrative, not actual model values.

2. Why Are Embeddings More Useful Than Token IDs?

- **Token IDs** are simply numerical identifiers and do not directly express meaning.
- **Embeddings** represent tokens as vectors that can learn semantic and linguistic relationships.
- Similar words may develop related representations during training.
- Transformer layers use embeddings to understand words in context.

For example, in *"AI models learn,"* the contextual representation of *models* can incorporate information from both *AI* and *learn*.

3. Label a Transformer Encoder



4. Identify Query, Key, and Value (Q, K, V)

Component	**Function**
Query (Q)	Searches for relevant information.
Key (K)	Helps determine how relevant another token is.
Value (V)	Carries information used to build contextual representations.

The token representations are transformed into Q, K, and V through learned linear projections.

5. Explain Which Words Receive Higher Attention

Consider the sentence **"AI models learn."**

- **AI ↔ models:** These words are related because AI describes the type of models.
- **models ↔ learn:** These words are related because models can learn.

When processing *models*, the Transformer may assign higher attention to *AI* or *learn*, depending on the context and learned attention weights.

Attention process:

Query and Key vectors produce attention scores. Softmax converts these scores into weights, which determine how strongly the corresponding Value vectors contribute to the output.

Note: These are possible relationships, not measured attention scores.

6. Compare Transformer Models

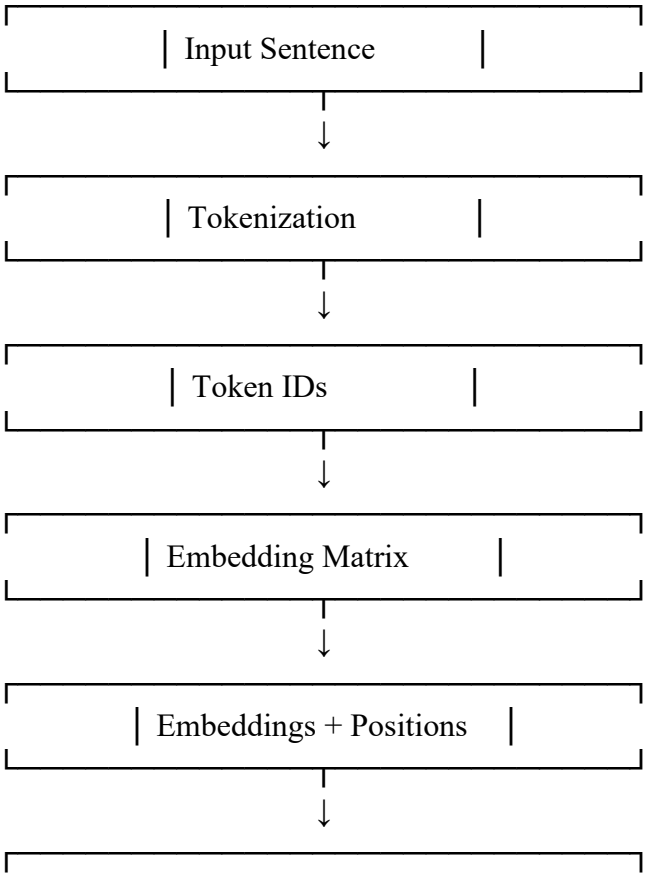
Model	Architecture	Main Application
BERT	Encoder-only	Text understanding and classification
GPT	Decoder-only	Text generation
T5	Encoder–decoder	Summarization and translation

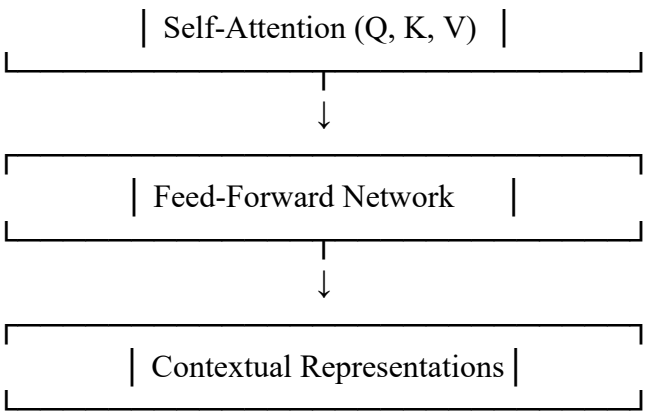
All three use token embeddings, but their architectures and attention mechanisms differ.

7. Complete the Transformer Pipeline

Input Sentence
↓
Tokenization
↓
Token IDs
↓
Embedding Lookup
↓
Positional Information
↓
Transformer Layers
↓
Contextual Representations
↓
Output Prediction

8. Block Diagram



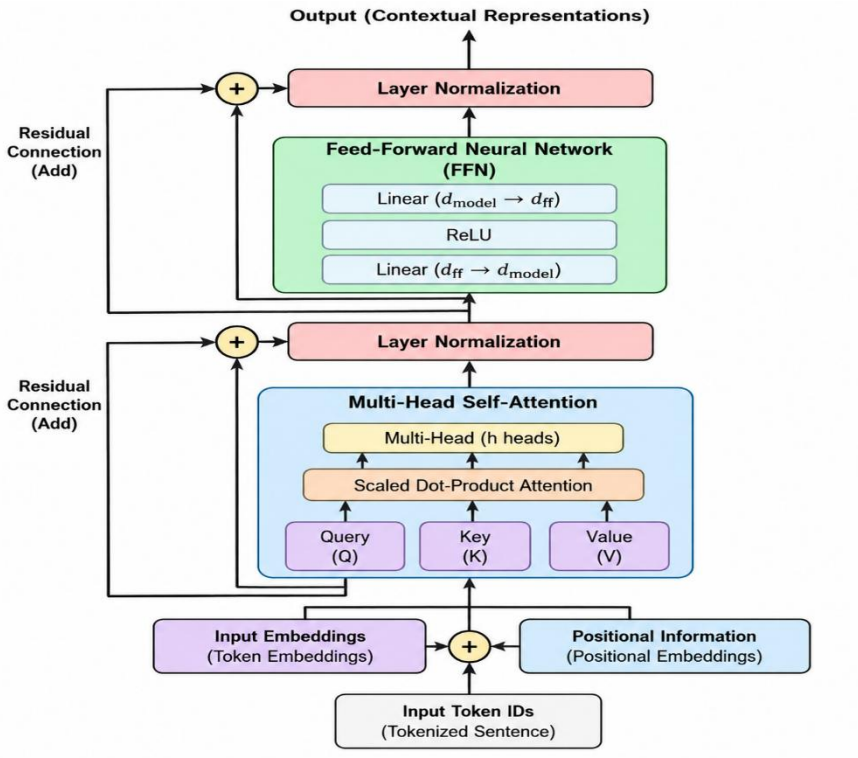


Conclusion

Embedding vectors convert token IDs into meaningful numerical representations. Unlike token IDs, embeddings can capture learned linguistic relationships. Transformer layers then use attention and contextual processing to understand how words relate to one another.

Question 4: Transformer Encoder Labelling

Topic: Transformer Encoder Architecture | **Bloom's Level:** Apply



1. Aim

To draw and explain a Transformer encoder block, identifying its major components and their functions.

2. Transformer Encoder Architecture

A **Transformer encoder** processes input tokens and generates contextual representations using self-attention and feed-forward neural networks.

3. Components of the Transformer Encoder

1. Input Embeddings + Positional Information

- Embeddings convert token IDs into numerical vectors.
- Positional information represents the order of tokens.

2. Multi-Head Self-Attention

Self-attention identifies relationships between words using three vectors:

Component	Function
Query (Q)	Searches for relevant information.
Key (K)	Helps calculate the relevance of other tokens.
Value (V)	Carries information used to form contextual representations.

Multiple attention heads can learn different relationships between tokens.

3. Residual Connections

Add the input of a sublayer to its output, helping preserve information and support training.

4. Layer Normalization

Normalizes activations to help stabilize the network during training.

5. Feed-Forward Neural Network (FFN)

Transforms each token's representation using linear layers and an activation function such as ReLU.

6. Contextual Representations

The encoder produces representations that incorporate information from the surrounding words.

4. Correct Order of Components

Input Embeddings + Positional Information

↓

Multi-Head Self-Attention

↓

Residual Connection + Layer Normalization

↓

Feed-Forward Neural Network

↓

Residual Connection + Layer Normalization

↓

Contextual Representations

5. Attention Weight Analysis

Consider the sentence **"AI models learn."**

- *AI* relates to *models* because it describes the type of models.
- *Models* relates to *learn* because models can learn.

When processing *models*, the encoder may assign higher attention to *AI* or *learn*, depending on the learned attention weights.

Attention scores are calculated using Query and Key vectors. Softmax converts the scores into weights, which determine how Value vectors contribute to the output.

Note: These relationships are illustrative, not measured attention scores.

6. Comparison of Transformer Models

Model	Architecture	Application
BERT	Encoder-only	Text understanding and classification
GPT	Decoder-only	Text generation
T5	Encoder–decoder	Summarization and translation

7. Transformer Pipeline

Input Sentence
↓
Tokenization
↓
Token IDs
↓
Embeddings + Positional Information
↓
Self-Attention (Q, K, V)
↓
Feed-Forward Network
↓
Contextual Representations
↓
Output / Downstream Task

8. Block Diagram

Use the **labelled Transformer encoder image provided above** to illustrate the complete architecture, including residual connections and layer normalization.

9. Conclusion

A Transformer encoder combines embeddings, positional information, multi-head self-attention, residual connections, layer normalization, and feed-forward networks to understand relationships between words and generate contextual representations.

Assessment Rubric Alignment

Criterion	How the answer addresses it
Mechanics Explanation (30%)	Explains the encoder components and their relationships.
Worked-Example Accuracy (25%)	Uses a relevant sentence to illustrate attention.
Mathematical Reasoning (20%)	Explains Q, K, V and attention weights.
Diagram & Working Clarity (15%)	Provides a labelled diagram and ordered pipeline.
Step Justification (10%)	Explains the purpose of each component.

Question 5: Query, Key, and Value Identification

Topic: Query–Key–Value | **Bloom's Level:** Analyze

1. Aim

To explain how Query, Key, and Value vectors are generated from token representations and how they identify relationships between words in a Transformer.

2. Tokenization and Token IDs

Given sentence:

"The student submitted the assignment because it was complete."

Word-level tokens:

["The", "student", "submitted", "the", "assignment", "because", "it", "was", "complete", "."]

Each token is converted into a numerical ID using a tokenizer's vocabulary. These IDs are then used to obtain embedding vectors.

3. Explain Embeddings

Embeddings are numerical vectors representing tokens. Positional information is added to represent word order.

For example, the representation of **"assignment"** contains learned token information and, after Transformer processing, can incorporate context from words such as *submitted* and *complete*.

4. Label a Transformer Encoder

Input Token IDs
↓
Embeddings + Positional Information
↓
Multi-Head Self-Attention
↓
Add & Layer Normalization
↓
Feed-Forward Neural Network
↓
Add & Layer Normalization
↓
Contextual Representations

5. How Query, Key, and Value Are Generated

The token representations are transformed using three learned weight matrices.

For an input representation X:

$$Q = XW_Q,\ K = XW_K,\ V = XW_V$$

Here, W_Q, W_K, and W_V are learned projection matrices.

Component	**Role**
Query (Q)	Represents what a token is looking for.
Key (K)	Provides information used to match a token with other tokens.
Value (V)	Carries the information that contributes to the output representation.

6. Explain Which Words Receive Higher Attention

Consider the phrase **"the assignment because it was complete."**

- **It ↔ assignment:** The word *it* may attend strongly to *assignment* because it is a likely reference.
- **It ↔ complete:** The word *complete* helps describe the referent.
- **Submitted ↔ assignment:** These words are related because the assignment is what was submitted.

Attention mechanism:

1. Queries are compared with Keys to calculate attention scores.
2. Softmax converts the scores into attention weights.
3. The weights are applied to the Value vectors.
4. The weighted Values are combined to produce contextual representations.

Note: These are plausible linguistic relationships, not measured attention weights.

7. Compare Transformer Models

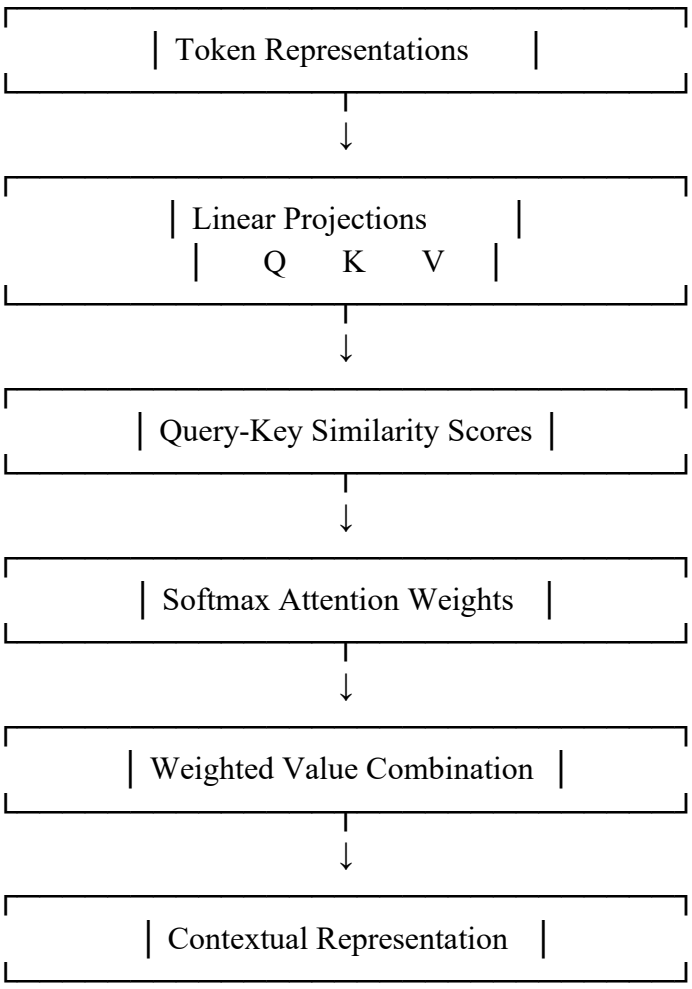
Model	Architecture	Main Application
BERT	Encoder-only	Text understanding
GPT	Decoder-only	Text generation
T5	Encoder–decoder	Summarization and translation

All three use attention mechanisms based on Query, Key, and Value, but their architectures and attention patterns differ.

8. Complete the Transformer Pipeline

Input Sentence
↓
Tokenization
↓
Token IDs
↓
Embeddings + Positional Information
↓
Query, Key, Value Projections
↓
Attention Scores and Weights
↓
Weighted Value Combination
↓
Feed-Forward Network
↓
Contextual Representations
↓
Output / Prediction

9. Block Diagram



10. Conclusion

Query, Key, and Value vectors are essential components of Transformer self-attention. Queries identify relevant information, Keys help calculate relevance, and Values provide the information combined to form contextual representations. This mechanism enables Transformers to learn relationships between words in a sentence.

Assessment Rubric Alignment

Criterion	**How the answer addresses it**
Mechanics Explanation (30%)	Explains Q, K, V and their relationships.
Worked-Example Accuracy (25%)	Applies the concepts to the given sentence.
Mathematical Reasoning (20%)	Includes projection equations and attention steps.
Diagram & Working Clarity (15%)	Provides labelled encoder and attention diagrams.
Step Justification (10%)	Explains the purpose of each component.

Question 6: Attention Weight Analysis

Topic: Self-Attention | **Bloom's Level:** Analyze

1. Aim

To analyze which words receive higher attention when processing a word and explain how attention weights help Transformers capture contextual relationships.

2. Tokenization and Token IDs

Given sentence:

"The cat chased the mouse because it moved quickly."

Word-level tokens:

["The", "cat", "chased", "the", "mouse", "because", "it", "moved", "quickly", "."]

Each token is converted into a numerical ID using the tokenizer's vocabulary.

3. Explain Embeddings

Embeddings convert token IDs into numerical vectors. Positional information is added to represent word order.

These vectors are processed by Transformer layers to learn contextual relationships between words.

4. Label a Transformer Encoder

Input Token IDs
↓
Embeddings + Positional Information
↓
Multi-Head Self-Attention
↓
Add & Layer Normalization
↓
Feed-Forward Network
↓
Add & Layer Normalization
↓
Contextual Representations

5. Identify Query, Key, and Value (Q, K, V)

Component	Function
Query (Q)	Searches for relevant information.
Key (K)	Helps determine the relevance of other tokens.
Value (V)	Carries information used to create contextual representations.

The model generates these vectors from token representations using learned projection matrices.

6. Which Words Receive Higher Attention?

Consider the sentence:

"The cat chased the mouse because it moved quickly."

When processing the word **"it,"** the following words may receive higher attention:

- **Mouse:** A likely reference for *it*, since the mouse is a plausible subject of *moved quickly*.
- **Moved:** Helps explain the action associated with *it*.
- **Quickly:** Provides additional information about how the movement occurred.

The word **cat** may also receive attention because it is another possible reference in the sentence.

How attention weights work:

1. The Query for *it* is compared with the Keys of other tokens.
2. The similarity scores are converted into attention weights using softmax.
3. The weights determine how much each token's Value contributes to the result.
4. The combined information produces a contextual representation of *it*.

Justification: Attention helps the Transformer connect words that are relevant to one another, supporting the interpretation of references and relationships across a sentence.

Note: These are reasonable linguistic interpretations, not actual measured attention weights. Exact weights depend on the model, layer, and attention head.

7. Compare Transformer Models

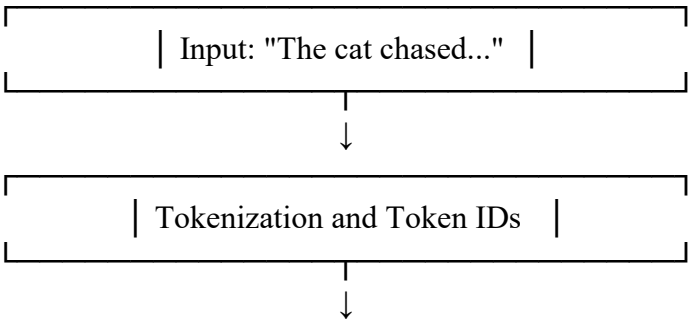
Model	Architecture	Main Application
BERT	Encoder-only	Text understanding
GPT	Decoder-only	Text generation
T5	Encoder–decoder	Summarization and translation

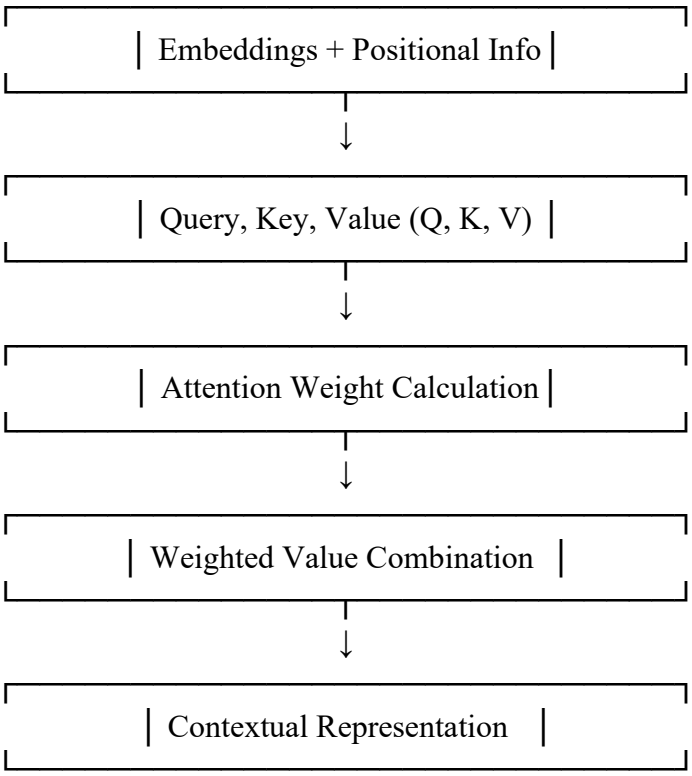
BERT can use bidirectional context, while GPT uses causal attention to process preceding tokens during generation.

8. Complete the Transformer Pipeline

Input Sentence
↓
Tokenization
↓
Token IDs
↓
Embeddings + Positional Information
↓
Query, Key, Value Projections
↓
Attention Scores → Softmax Weights
↓
Weighted Value Combination
↓
Feed-Forward Network
↓
Contextual Representations

9. Block Diagram





10. Conclusion

Attention weights help Transformers determine how strongly words influence one another. In this sentence, *mouse*, *moved*, and *quickly* may be relevant when processing *it*. By combining information from related tokens, self-attention helps the model build contextual representations and interpret sentence meaning.

Assessment Rubric Alignment

Criterion	**How the answer addresses it**
Mechanics Explanation (30%)	Explains self-attention and contextual relationships.
Worked-Example Accuracy (25%)	Applies attention analysis to the given sentence.
Mathematical Reasoning (20%)	Explains attention scores, softmax, and weighted Values.
Diagram & Working Clarity (15%)	Includes labelled diagrams and the processing pipeline.
Step Justification (10%)	Justifies why particular words may receive higher attention.

Question 7: Comparing Transformer Models

Topic: Transformer Model Comparison | **Bloom's Level:** Analyze

1. Aim

To compare BERT, GPT, and T5 based on their attention mechanisms, input-output behaviour, and applications, and identify the most suitable model for different NLP tasks.

2. Tokenization and Token IDs

Example sentence: "AI transforms language."

Word-level tokens:

["AI", "transforms", "language", "."]

Each token is converted into a numerical ID using the model's tokenizer. The actual IDs depend on the vocabulary used by each model.

3. Explain Embeddings

Embeddings convert token IDs into numerical vectors that represent learned token information. Positional information helps represent token order.

These vectors are processed by Transformer layers to capture contextual relationships between words.

4. Transformer Encoder Labelling

Input Token IDs
↓
Embeddings + Positional Information
↓
Multi-Head Self-Attention
↓
Add & Layer Normalization
↓
Feed-Forward Network
↓
Add & Layer Normalization
↓
Contextual Representations

This is a simplified **Transformer encoder** structure, commonly associated with BERT.

5. Identify Query, Key, and Value (Q, K, V)

Component	Function
Query (Q)	Searches for relevant information.
Key (K)	Helps determine the relevance of other tokens.
Value (V)	Carries information used to build contextual representations.

All three models use attention mechanisms based on Query, Key, and Value, although their attention patterns differ.

6. Explain Which Words Receive Higher Attention

Consider the sentence **"AI transforms language."**

- **AI ↔ transforms:** These words are related because AI performs the action.
- **Transforms ↔ language:** These words are related because language is the object of the action.

When processing *transforms*, the model may assign higher attention to *AI* or *language*, depending on the context and learned weights.

Attention process: Query–Key scores are calculated, softmax converts them into attention weights, and the weights determine how the Value vectors contribute to the output.

Note: These are illustrative relationships, not measured attention scores.

7. Compare BERT, GPT, and T5

Feature	**BERT**	**GPT**	**T5**
Architecture	Encoder-only	Decoder-only	Encoder–decoder
Attention approach	Bidirectional self-attention	Causal self-attention	Encoder self-attention, decoder causal attention, and cross-attention
Input-output behaviour	Text → contextual representations or labels	Prompt → generated text	Input text → output text
Main purpose	Language understanding	Text generation	Text-to-text processing
Applications	Classification, entity recognition	Dialogue, content generation	Summarization, translation

8. Select the Most Suitable Model

(a) Text Classification — BERT

BERT is suitable because its bidirectional attention captures information from both sides of a token. Its contextual representations can be used to classify text into categories such as positive or negative sentiment.

(b) Open-Ended Text Generation — GPT

GPT is suitable because its causal attention predicts the next token based on preceding tokens. Repeating this process enables it to generate coherent text.

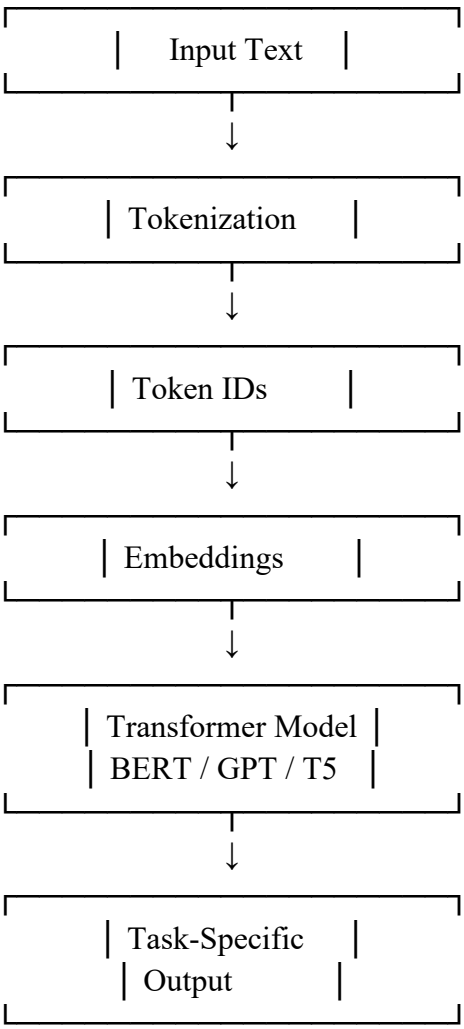
(c) Text Summarization — T5

T5 is suitable because its encoder processes the input document and its decoder generates a shorter output. This encoder–decoder structure is well suited to text transformation tasks.

9. Complete the Transformer Pipeline

Input Text
↓
Tokenization
↓
Token IDs
↓
Embeddings + Positional Information
↓
Transformer Layers
↓
Contextual Representations
↓
Task-Specific Output
↓
Classification / Generated Text / Summary

10. Block Diagram



11. Conclusion

BERT, GPT, and T5 use Transformer attention mechanisms but differ in architecture and purpose. **BERT is suitable for text classification, GPT for open-ended text generation, and T5 for text summarization.** Selecting the appropriate model depends on the input-output requirements of the task.

Assessment Rubric Alignment

Criterion	How the answer addresses it
Mechanics Explanation (30%)	Explains architectures and attention approaches.
Worked-Example Accuracy (25%)	Uses a sentence to illustrate tokenization and attention.
Mathematical Reasoning (20%)	Explains the Query–Key–Value attention process.
Diagram & Working Clarity (15%)	Includes encoder and pipeline diagrams.
Step Justification (10%)	Justifies the model selected for each application.

Question 8: Completing the Transformer Pipeline

Topic: Transformer Pipeline | **Bloom's Level:** Apply

1. Aim

To complete the Transformer processing pipeline and understand how input text is converted into tokens, processed by a Transformer, and used to predict the next token.

2. Completed Transformer Pipeline

The completed pipeline is:

Text Input
↓
Tokenization
↓
Token-to-ID Conversion
↓
Embedding and Positional Information
↓
Transformer Layers (Self-Attention and Feed-Forward Network)
↓
Output Probabilities
↓
Predicted Token

3. Function of Each Stage

1. Text Input

The original sentence or prompt provided by the user.

Example: "AI makes learning"

2. Tokenization

The input text is divided into smaller units called tokens, which may be words, subwords, or punctuation.

Example: ["AI", "makes", "learning"]

3. Token-to-ID Conversion

Each token is mapped to a numerical ID using the tokenizer's vocabulary. The model processes these IDs rather than raw text.

Example: [12, 45, 78]

Note: These IDs are illustrative.

4. Embedding and Positional Information

Token IDs are converted into numerical vectors called embeddings. Positional information helps the model understand the order of tokens in the sequence.

5. Transformer Layers

The Transformer processes the vectors using self-attention and feed-forward networks. Self-attention identifies relationships between tokens, while feed-forward networks further transform their representations.

6. Output Probabilities

The model converts its output representations into probabilities for possible next tokens. For example, after "AI makes learning," it might assign probabilities to words such as "easier," "interesting," or "accessible."

7. Predicted Token

A token is selected based on the output probabilities and the model's decoding strategy. The selected token can be added to the sequence to generate further text.

4. Worked Example

Suppose the input is:

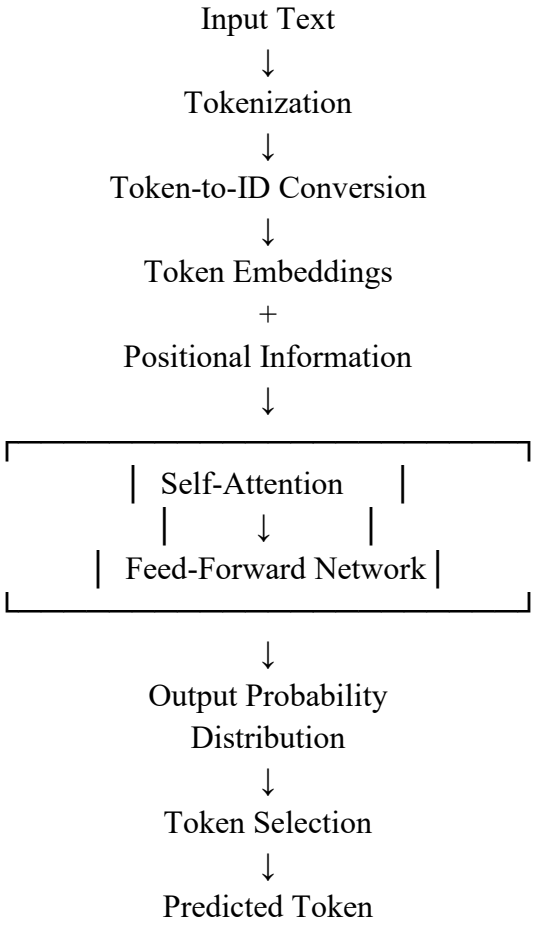
“AI makes learning”

Stage	**Example output**
Text Input	AI makes learning
Tokenization	["AI", "makes", "learning"]
Token-to-ID Conversion	[12, 45, 78]
Embedding and Positional Information	Vectors representing tokens and their positions
Transformer Layers	Context-aware token representations
Output Probabilities	easier: 0.65, fun: 0.25, accessible: 0.10
Predicted Token	easier

Note: The token IDs and probabilities above are hypothetical examples, not actual model outputs.

The resulting text becomes **“AI makes learning easier.”**

5. Transformer Block Diagram



Important: This pipeline describes a typical autoregressive text-generation process. Encoder-only Transformers, such as BERT, commonly produce contextual representations for tasks like classification rather than predicting the next token.

6. Outcome

The Transformer pipeline converts text into tokens and numerical representations, processes contextual relationships through Transformer layers, and predicts a suitable output token using a probability distribution.

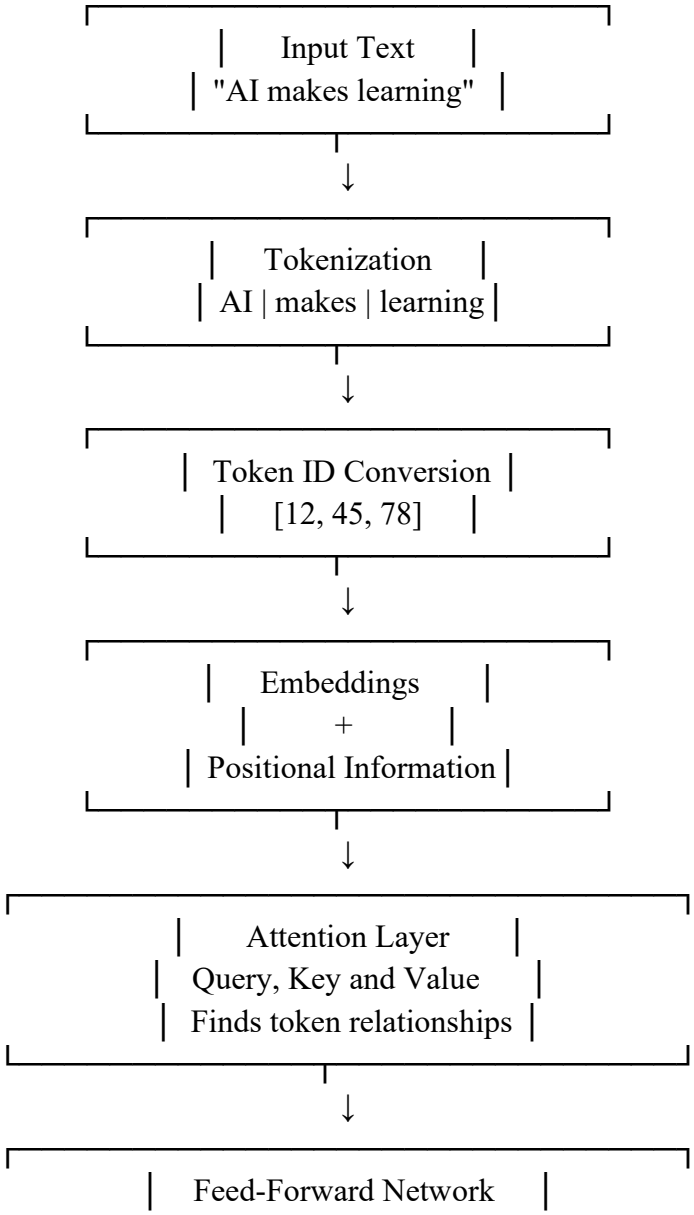
Question 9: Transformer Block Diagram

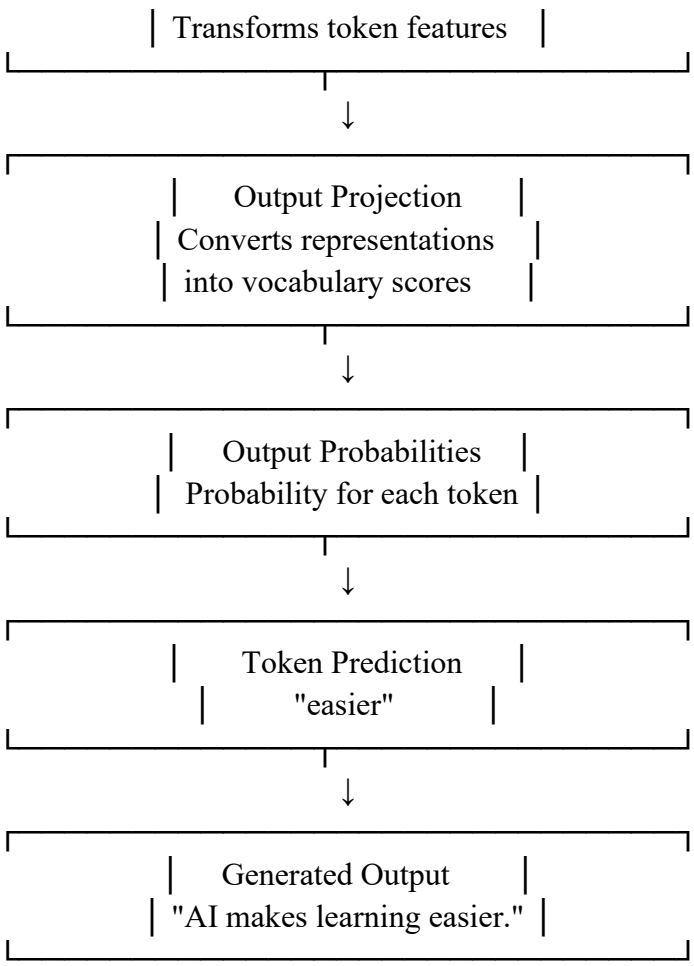
Topic: Transformer Architecture | **Bloom's Level:** Create

1. Aim

To draw a block diagram illustrating how a Transformer-based language model processes an input sentence and generates an output through tokenization, token ID conversion, embeddings, attention, feed-forward layers, and token prediction.

2. Block Diagram





Note: The token IDs and predicted word are illustrative. A real model may use subword tokens and different numerical IDs.

3. Function of Each Block

1. **Input Text:** Receives the sentence or prompt from the user.
2. **Tokenization:** Divides the sentence into smaller units called tokens.
3. **Token ID Conversion:** Converts each token into a numerical ID using the tokenizer's vocabulary.
4. **Embeddings and Positional Information:** Converts token IDs into vector representations and incorporates information about token positions.
5. **Attention Layer:** Uses Query, Key, and Value representations to determine relationships between tokens and identify relevant context.
6. **Feed-Forward Network:** Processes and transforms the representations produced by the attention layer.
7. **Output Projection:** Maps the final hidden representation to scores for possible vocabulary tokens.
8. **Output Probabilities:** Converts the scores into a probability distribution over possible next tokens.
9. **Token Prediction:** Selects a token according to the model's decoding strategy.
10. **Generated Output:** Adds the predicted token to the text. The process can repeat until the model generates an appropriate stopping token or reaches its output limit.

Technical note: A standard Transformer block also includes residual connections and layer normalization. In a decoder-only language model, self-attention is causal, meaning each position can attend only to permitted preceding positions. During generation, the model repeats the prediction process

to produce subsequent tokens.

4. Outcome

The block diagram demonstrates the complete flow of a Transformer-based language model, from input text to predicted tokens and generated output. It highlights how attention captures contextual relationships, feed-forward layers transform representations, and output projection enables token prediction.

Question 10: Integrated Transformer Mechanics

Topic: Integrated Transformer Pipeline | **Bloom's Level:** Analyze

1. Aim

To trace the sentence "Generative AI creates new content." through the Transformer pipeline and analyze how self-attention identifies relationships between words to generate meaningful output.

2. Tokenization

Tokenization divides the input sentence into smaller units called tokens.

Input sentence:

"Generative AI creates new content."

Illustrative word-level tokens:

["Generative", "AI", "creates", "new", "content", "."]

A real Transformer tokenizer may divide some words into smaller subword tokens.

3. Token-to-ID Conversion

Each token is mapped to a numerical ID using the tokenizer's vocabulary.

Token	Illustrative token ID
Generative	105
AI	42
creates	218
new	67
content	154
.	9

Token ID sequence:

[105, 42, 218, 67, 154, 9]

Note: These IDs are examples only. Actual IDs depend on the tokenizer and its vocabulary.

Token IDs allow the model to process text numerically rather than as ordinary characters or words.

4. Embedding Generation

The token IDs are used to retrieve corresponding vectors from the model's embedding matrix. Positional information is also incorporated so the model can represent the order of tokens.

For example, the embeddings represent:

- **Generative:** The concept of generating or creating something.
- **AI:** Artificial intelligence, the subject of the sentence.
- **creates:** The action performed.

- **new:** A property describing the content.
- **content:** The object being created.

These meanings are simplified interpretations. Actual embeddings are learned numerical vectors and do not necessarily represent individual words as explicit dictionary definitions.

5. Self-Attention Mechanism

Self-attention helps the model determine which tokens are relevant to one another when processing the sentence.

Each token representation is transformed into three vectors:

- **Query (Q):** Represents what information a token is looking for.
- **Key (K):** Represents information used to determine how relevant a token is to another token.
- **Value (V):** Contains the information that contributes to the updated representation.

The attention calculation is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Here, d_k is the dimension of the key vectors. The softmax function converts attention scores into normalized weights, which determine how the value vectors are combined.

Example of Word Relationships

For the sentence "Generative AI creates new content," attention may help the model identify the following relationships:

Word	Relevant relationship	Explanation
Generative	AI	Connects the type of AI being discussed.
AI	creates	Connects the subject to its action.
creates	content	Identifies what is being created.
new	content	Connects the descriptive word to the object.
content	creates	Connects the object to the action that produces it.

These are intuitive examples of relevant relationships, **not measured attention weights**. Actual attention patterns depend on the model, layer, head, and input context.

6. Contextual Representation

After self-attention, the model combines information from relevant tokens to build contextual representations. Feed-forward networks further transform these representations, with residual connections and layer normalization used in standard Transformer blocks.

For example:

- **AI** is interpreted in relation to the action *creates*.
- **Creates** is understood in relation to the subject *Generative AI* and the object *content*.
- **New** is interpreted as describing *content*.

Thus, the model processes the sentence as a connected sequence rather than treating every word independently.

7. Output Prediction

For an autoregressive language model such as GPT, the final representation at the last input position is projected into vocabulary scores. These scores are converted into probabilities for possible next tokens.

For example, after receiving:

“Generative AI creates new content.”

The model might assign hypothetical probabilities to possible continuations:

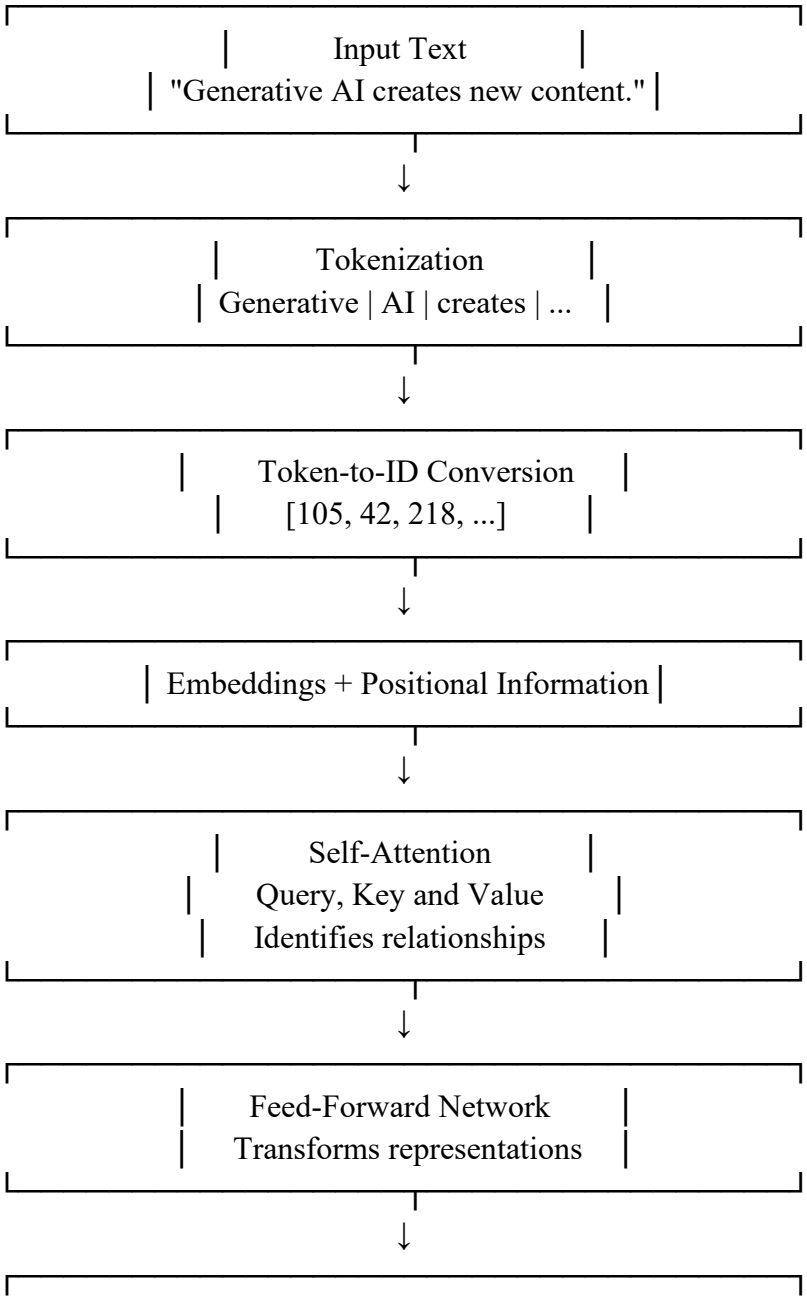
Possible next token	Illustrative probability
efficiently	0.50
automatically	0.30
creatively	0.20

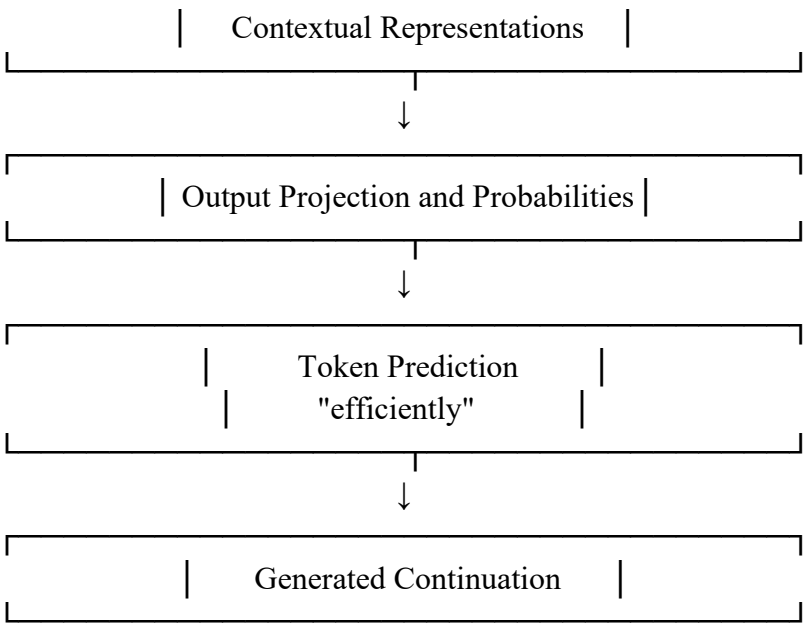
The model selects a token according to its decoding strategy. If it selects **“efficiently,”** the generated sentence becomes:

“Generative AI creates new content efficiently.”

Note: These probabilities are hypothetical, not actual model outputs. The predicted continuation depends on the model and decoding settings.

8. Integrated Transformer Pipeline





9. Analysis: How Attention Improves Understanding

Self-attention enables the model to connect words according to their relevance within a sentence. It helps relate the subject *AI* to the action *creates*, connect the action to its object *content*, and associate the adjective *new* with the appropriate noun.

Without contextual processing, the model would have less information about how words relate to one another. By combining relevant token representations, self-attention helps the Transformer build contextual information that supports more coherent and appropriate predictions.

Key point: Attention does not independently guarantee human-like understanding. It is a mechanism for learning and using relationships between token representations.

10. Outcome

The sentence was traced through all major stages of the Transformer pipeline: tokenization, token ID conversion, embedding generation, self-attention, contextual representation, and output prediction. The analysis demonstrates that **self-attention helps a Transformer use relationships between words to construct contextual representations and generate suitable text continuations.**